

1-G-Coin Problem

Problem Statement

Write a program to take value V and we want to make change for V Rs, and we have infinite supply of each of the denominations in Indian currency, i.e., we have infinite supply of {1, 2, 5, 10, 20, 50, 100, 500, 1000} valued coins/notes, what is the minimum number of coins and/or notes needed to make the change.

C Code

```
#include <stdio.h>

int main() {
    int v, count = 0;
    int coins[] = {1000, 500, 100, 50, 20, 10, 5, 2, 1};

    scanf("%d", &v);

    for (int i = 0; i < 9; i++) {
        count += v / coins[i];
        v %= coins[i];
    }

    printf("%d", count);

    return 0;
}
```

Sample Input

64

Sample Output

4

2-G-Cookies Problem

Problem Statement

Assume you are an awesome parent and want to give your children some cookies. But, you should give each child at most one cookie.

Each child i has a greed factor $g[i]$, which is the minimum size of a cookie that the child will be content with; and each cookie j has a size $s[j]$. If $s[j] \geq g[i]$, we can assign the cookie j to the child i , and the child i will be content. Your goal is to maximize the number of your content children and output the maximum number.

C Code

```
#include <stdio.h>

void sort(int a[], int n) {
    for (int i = 0; i < n - 1; i++)
        for (int j = 0; j < n - i - 1; j++)
            if (a[j] > a[j + 1]) {
                int t = a[j];
                a[j] = a[j + 1];
                a[j + 1] = t;
            }
}

int main() {
    int n, m, i = 0, j = 0, count = 0;

    scanf("%d", &n);
    int g[n];
    for (i = 0; i < n; i++) scanf("%d", &g[i]);

    scanf("%d", &m);
    int s[m];
    for (i = 0; i < m; i++) scanf("%d", &s[i]);

    sort(g, n);
    sort(s, m);

    i = 0;
    j = 0;

    while (i < n && j < m) {
        if (s[j] >= g[i]) {
            count++;
            i++;
        }
        j++;
    }

    printf("%d", count);

    return 0;
}
```

Sample Input

```
3
1 2 3
2
1 1
```

Sample Output

```
1
```

3-G-Burger Problem

Problem Statement

A person needs to eat burgers. Each burger contains a count of calorie. After eating the burger, the person needs to run a distance to burn out his calories. If he has eaten i burgers with c calories each, then he has to run at least $3^i * c$ kilometers to burn out the calories.

He can eat burgers in any order. Determine the minimum distance he needs to run to burn out the calories. Use an efficient sorting algorithm and apply greedy approach.

C Code

```
#include <stdio.h>

void sort(int a[], int n) {
    for (int i = 0; i < n - 1; i++)
        for (int j = 0; j < n - i - 1; j++)
            if (a[j] < a[j + 1]) {
                int t = a[j];
                a[j] = a[j + 1];
                a[j + 1] = t;
            }
}

int main() {
    int n;
    long long ans = 0, power = 1;

    scanf("%d", &n);
    int a[n];

    for (int i = 0; i < n; i++)
        scanf("%d", &a[i]);

    sort(a, n);

    for (int i = 0; i < n; i++) {
        ans += power * a[i];
        power *= 3;
    }

    printf("%lld", ans);

    return 0;
}
```

Sample Input

```
3
1 3 2
```

Sample Output

```
18
```

4-G-Array Sum Max Problem

Problem Statement

Given an array of N integers, we have to maximize the sum of $arr[i] * i$, where i is the index of the element ($i = 0, 1, 2, \dots, N-1$). Write an algorithm based on Greedy technique with a complexity $O(n \log n)$.

C Code

```
#include <stdio.h>
#include <stdlib.h>

int compare(const void *a, const void *b) {
    return (*(int *)a - *(int *)b);
}

int main() {
    int n;
    long long sum = 0;

    scanf("%d", &n);
    int a[n];

    for (int i = 0; i < n; i++)
        scanf("%d", &a[i]);

    qsort(a, n, sizeof(int), compare);

    for (int i = 0; i < n; i++)
        sum += (long long)a[i] * i;

    printf("%lld", sum);

    return 0;
}
```

Sample Input

```
5
2 5 3 4 0
```

Sample Output

```
40
```

5-G-Product of Array Elements-Minimum

Problem Statement

Given two arrays `array_One[]` and `array_Two[]` of same size `N`. We need to first rearrange the arrays such that the sum of the product of pairs (1 element from each) is minimum. That is $\text{SUM } (A[i] * B[i])$ for all `i` is minimum.

C Code

```
#include <stdio.h>
#include <stdlib.h>

int compare(const void *a, const void *b) {
    return (*(int *)a - *(int *)b);
}

int main() {
    int n;
    long long sum = 0;

    scanf("%d", &n);

    int a[n], b[n];

    for (int i = 0; i < n; i++)
        scanf("%d", &a[i]);

    for (int i = 0; i < n; i++)
        scanf("%d", &b[i]);

    qsort(a, n, sizeof(int), compare);
    qsort(b, n, sizeof(int), compare);

    for (int i = 0; i < n; i++)
        sum += (long long)a[i] * b[n - 1 - i];

    printf("%lld", sum);

    return 0;
}
```

Sample Input

```
3
1
2
3
4
5
6
```

Sample Output

```
28
```